

Point-in-Time Feature History

How the Snowflake Feature Store tracks feature values over time — and how to serve a multi-year (e.g., 2-year) history.

1. Two stores, two jobs

A feature view can be served from two places, and only one of them keeps history:

- **Online store** (Postgres-backed in this deployment): holds the **latest value per entity only**. Each sync overwrites the prior value. Built for millisecond point lookups at inference time — **not** a history mechanism.
- **Offline store** (Snowflake tables / dynamic tables): where feature history lives and where **point-in-time correctness** is computed. This is the layer that answers “what was this feature on a date 14 months ago?”

2. How much history is retained depends on the feature-view type

Feature view type	History retained	Online?	How it is built
Standard batch	Latest only — each refresh overwrites	Yes	refresh_freq; overwrites snapshot
Append-only batch	Full time series — appends values + timestamp each refresh	Yes	append_only=True, timestamp_col, cron refresh_freq; backup_source to backfill
Time-windowed / stream	Event-time-indexed tiles (trailing windows)	Yes	Aggregations API (feature_granularity) / StreamSource
Iceberg-backed	Full, open table format	No	StorageConfig on an external volume

Append-only batch feature views are the purpose-built mechanism for “show how a feature changed over time.” Every scheduled refresh appends the current values alongside a timestamp to a managed snapshot table, accumulating a complete time series. Requires snowflake-ml-python 1.41+, a timestamp_col, and a cron refresh_freq.

3. Reconstructing a value “as of” any past moment

History is not queried by hand. The Feature Store performs an **ASOF join**: for each row in your spine (entity + timestamp), it selects the most recent feature snapshot at or before that timestamp, so no future data leaks into the result.

```
spine row: (entity=ACCT_42, ts=2024-08-01)
snapshots: 2024-06-15 -> score 0.12
2024-07-20 -> score 0.31 <= chosen (most recent ≤ ts)
2024-09-03 -> score 0.55 (ignored: after ts)
```

APIs that apply point-in-time logic via spine_timestamp_col:

- fs.generate_training_set(...) / fs.generate_dataset(...) — build point-in-time-correct training data.

- `fs.retrieve_feature_values(...)` — fetch the values that were current as of each timestamp (scoring or historical inspection).

4. Serving a 2-year history

The length of the window is governed by the **physical retention of the timestamped rows**, not by any platform cap:

- **Append-only batch FV**: retain ~2 years of the snapshot table (a daily cadence is roughly 730 rows per entity).
- **Time-windowed / stream FV**: retain ~2 years of the underlying timestamped source events.
- **Periodic snapshots** of a standard FV are a lower-fidelity alternative if append-only is not available.

Important: Time Travel is **not** the mechanism here. It maxes out at 90 days and is for recovering accidental changes — not for feature history. Long-horizon history must come from feature data persisted **with timestamps** (append-only FV or retained timestamped sources). Storage grows with retention, so size for it; storage cost is typically small relative to the analytical value.

5. Mapping to this demo's feature views

- **ACCOUNT_PROFILE** — standard batch online FV today, so it keeps the latest profile only. To give 2 years of profile history, register it with `append_only=True`, a daily cron refresh, and a timestamp column (use `backup_source` to seed existing history).
- **ACCOUNT_VELOCITY** — stream FV with continuous time-windowed aggregations keyed by `EVENT_TS`. It is already event-time indexed; history depth equals the retention of the `TRANSACTION_EVENTS` source.
- **TXN_RISK_SIGNALS** — real-time FV computed at read time from per-request inputs. By design it is not stored, so it carries no history.

6. Recommendation

For a “feature values over a 2-year period” requirement: use an **append-only batch feature view** (or retain 2 years of timestamped events for windowed features), then reconstruct any point in time with `generate_dataset(spine_timestamp_col=...)`. The live demo path stays unchanged — an append-only variant can be added as a new feature-view version to produce a visible “feature drift over time” artifact without touching the existing flow.